

RAG vs Fine-tuning vs Prompt-tuning

Four levers to make an LLM do your job - and when to pull each - 2026-07-16



THE MENTAL MODEL

Four levers. Each touches a different part of the stack:

- **Prompt eng.** - change the input *text*. Model unchanged.
- **RAG** - change the input *context*, fetching facts at runtime. Model unchanged.
- **Fine-tune** - change the model *weights*. Model itself changes.
- **Prompt-tune** - learn a small *prompt vector*, weights frozen.

The rule: start with prompt engineering → add RAG for knowledge → fine-tune only when evidence says the other two can't. Real stacks combine all three.

1 · PROMPT ENGINEERING

Structure the input - role, instructions, few-shot examples, output format, chain-of-thought.

- Cheapest, fastest. Hours to days, ~\$0.
- No infra, no training, no data pipeline.
- **Ceiling:** limited to what's already in the weights + what fits the context window.
- **Best for:** behavior, format, tone, reasoning scaffolds, feasibility checks.

2 · RAG

At query time, fetch relevant docs and inject them into the prompt - answer from **your** data.

Pipeline

chunk → embed → store (vector DB)
query → embed → search → top-K
→ rerank → prompt → generate

What actually matters

- Chunk ~**400-512 tokens**, **10-20% overlap** (default).
- **Same embedder** for index AND query - else broken search.
- **Hybrid search** (vector + keyword) is the standard.
- **Rerank** = the production-grade step.
- Eval with **RAGAS**; golden set 50-100 Q&A.

Best for: fresh, proprietary, or **citable** knowledge. Update by re-indexing - no retraining.

3 · FINE-TUNING

Continue training on your examples so **weights shift**. The only lever that changes the model.

Method	Memory / cost
Prompt/prefix tune	Tiny
LoRA	10-20x less than full
QLoRA (4-bit)	7B on a \$1.5k GPU
Full FT	7B ≈ 100-120GB VRAM

- **PEFT** = LoRA/QLoRA/adapters; ~90-95% of full quality.
- LoRA ~\$13 vs full ~\$322 for the same run.
- Earns its cost only at **scale** + deep specialization. Premature before PMF.

Trap: teaches skill, not facts. Don't fine-tune to inject changing knowledge - it stales and can't cite.

4 · PROMPT-TUNING

Learn continuous **prompt vectors** (not readable text), base weights frozen. Between prompt eng. and full FT.

- **Prompt tuning** - vectors at the input embedding only.
- **Prefix tuning** - tensors at every transformer block.
- **P-tuning** - vectors anywhere, via a prompt encoder.

With enough training data, beats hand-written prompts while touching almost no params. [\[x\]](#)
[prompt engineering](#)

DECIDE IN 4 LINES

WORDING / FORMAT / REASONING
→ Prompt engineering (always start)
CURRENT / PRIVATE / CITABLE facts
→ RAG
Fixed SKILL / STYLE at scale
→ Fine-tune (LoRA/QLoRA first)
Have data, want max perf, min ops
→ Prompt-tuning

Escalation ladder (cheapest first)

1. Prompt (hrs) → 2. RAG (days, \$70-1k/mo) → 3. PEFT/LoRA (small \$) → 4. Full FT (months, \$\$\$).

AT A GLANCE

	Prompt	RAG	Fine-tune
Changes model?	No	No	Yes
Adds facts?	No	Yes	Poorly
Time to value	Hours	Days-wks	Wks-mos
Cost	~\$0	\$	\$\$-\$\$\$
Can cite?	No	Yes	No
Update by	Edit text	Re-index	Retrain
Ops burden	None	Medium	High

DON'T GET BURNED

- X "Fine-tune to add company knowledge." → Use RAG.
- X "RAG vs FT is either/or." → They compose.
- X "Fine-tune to look serious." → Burned budget pre-PMF.
- X "Prompt tuning = prompt engineering." → No: vectors vs text.
- X Mismatched embedders → silently broken retrieval.

IF YOU REMEMBER ONE THING
Fine-tuning teaches a **skill**; RAG supplies **facts**. Default to prompt + RAG - fine-tune only the few pipelines that prove they need it.